openagent-frontend

> **The OpenAgent user interface** — the lean Streamlit chat UI that sits in front of `openagent-api`.

Overview

`openagent-frontend` is the **user interface layer** of the OpenAgent system. It is a lean Streamlit web app that renders the chat experience, tracks in-session conversation state, and consumes a streaming response from `openagent-api`. That is the entire job.

This repo is scoped to the UI only. It has no model, no persona, no inference code, no database, no auth backend. It does not own the agent identity — the persona is owned upstream by `openagent-api`. What lives here:

- The **Streamlit chat UI** that users talk to
- The **HTTP/SSE client** that streams responses from `openagent-api`
- The **JSON ChatCompletion chunk decoder** (`sse_decoder.py`) that turns the byte stream into typed events
- The **conversation state** held per browser session
- The **health gate** that locks the UI until the upstream model is ready
- The **error display** with the emoji prefixes (🔌 ⏳ 🔐 ⚠❌)

I kept the boundary with `openagent-api` deliberately sharp: the frontend owns *how the agent looks and feels to the user*, the gateway owns *who the agent is, how requests are authenticated, and how the upstream stream is relayed*. The two communicate over a stable HTTP/SSE contract.

Where This Fits

```
openagent-os
│
├── openagent-infra      ← separate repo
│   └── Model proxy → BYOC Provider (port 8002)
│       Stateless proxy that forwards to compute providers
│
├── openagent-api        ← separate repo
│   └── FastAPI gateway (port 8001)
│       Owns the persona, auth chain, and SSE relay
│
├── openagent-frontend   ← YOU ARE HERE
│   └── Streamlit chat UI (port 8000)
│       Pure UI layer. Talks only to openagent-api.
│
└── openagent-logger     ← separate repo
    └── Structured event log (called by openagent-api)
```

The naming convention is intentional:

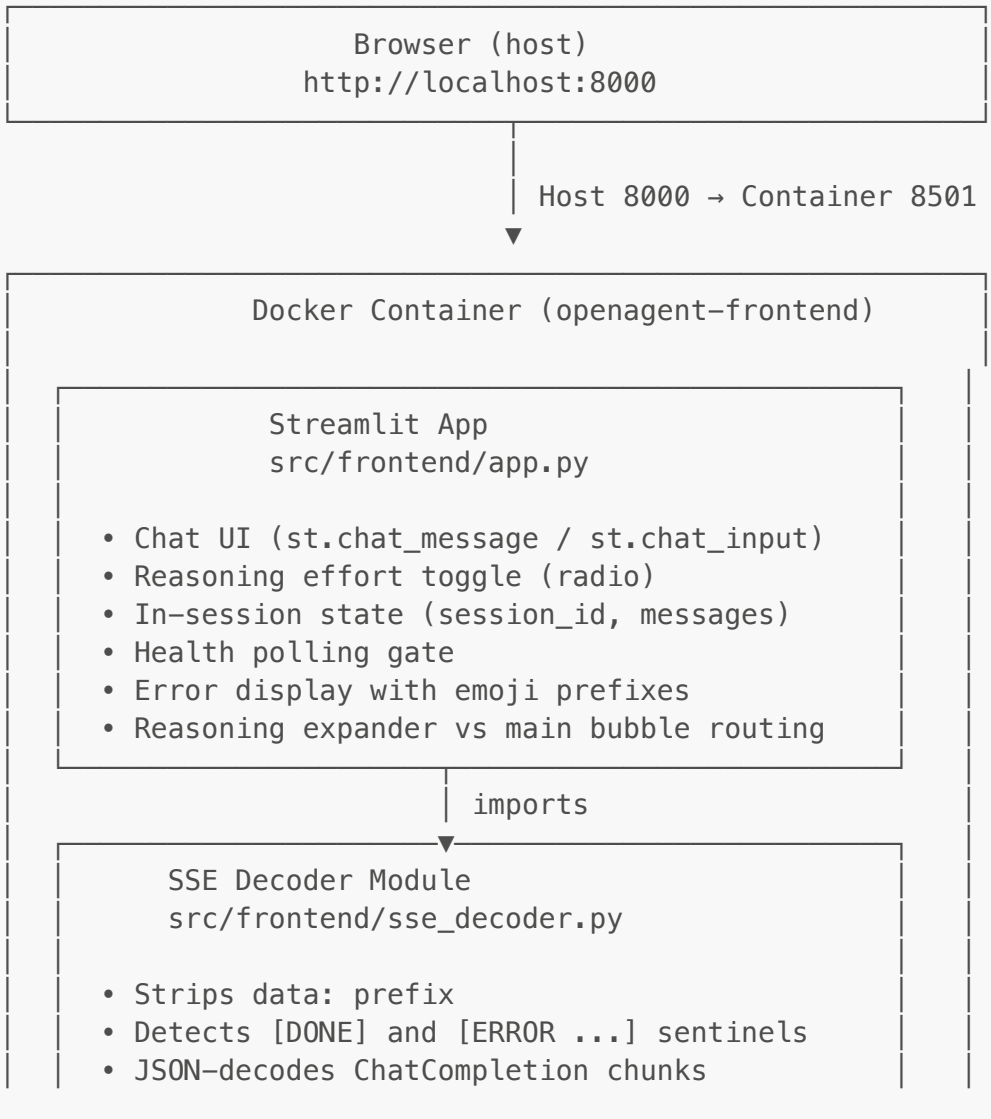
- `openagent-infra` handles the **model** connectivity and compute provision.
- `openagent-*` (api, frontend, logger) handle the **product** — gateway, UI, identity, and state.

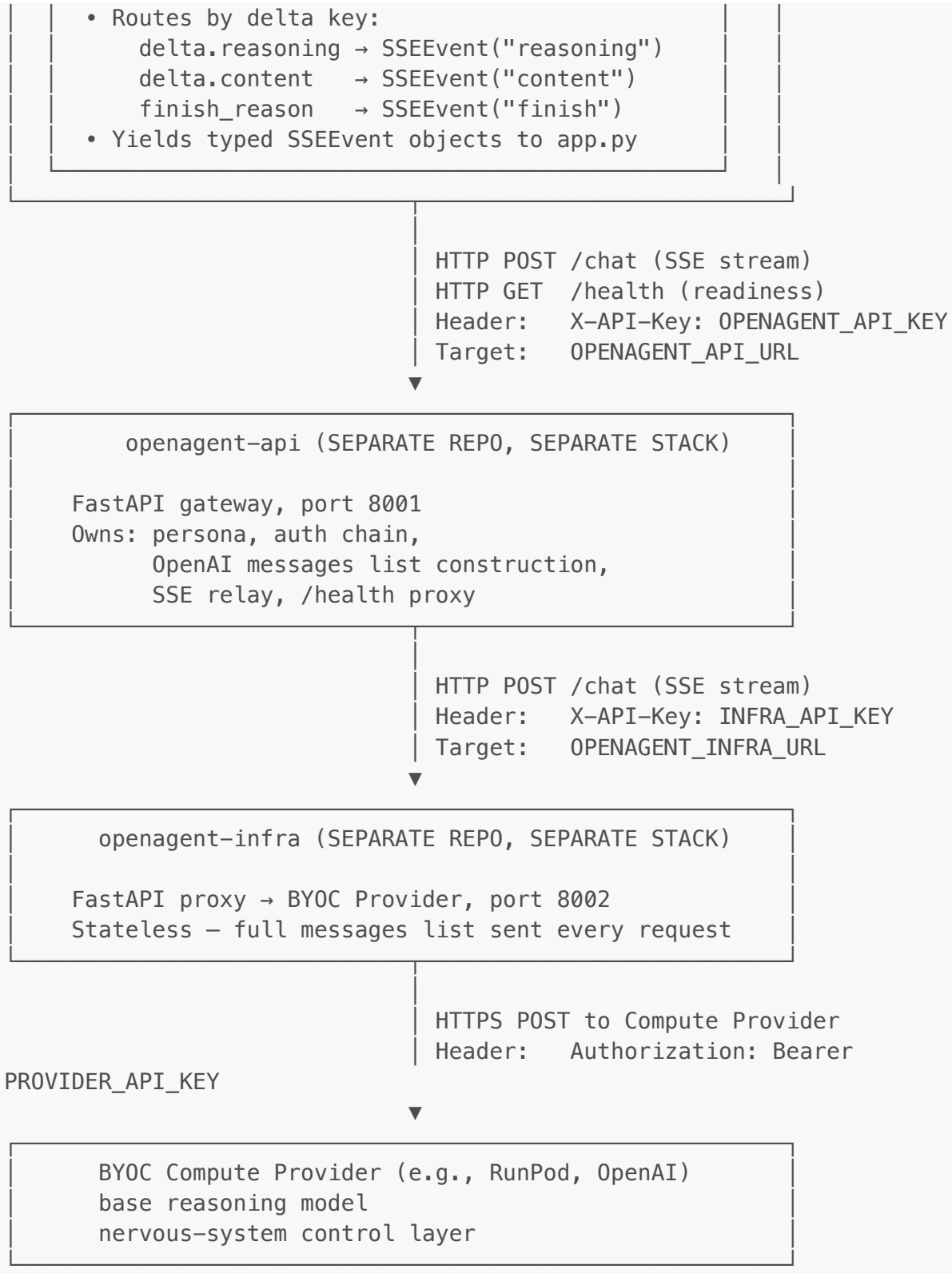
Port topology:

```
User → openagent-frontend (:8000) → openagent-api (:8001) → openagent-infra (:8002) → BYOC Provider
```

Users only ever interact with port 8000. Port 8001 is `openagent-api`, port 8002 is `openagent-infra`, and the compute provider is reached over HTTPS — none of those layers are exposed to end users. `openagent-frontend` is the only client of `openagent-api`, and `openagent-api` is the only thing `openagent-frontend` talks to.

Architecture





Request flow

1. User types a message in the Streamlit chat input.
2. The frontend appends it to `st.session_state.messages`.
3. The frontend constructs a user/assistant-only messages list:

```
[
  { "role": "user",      "content": "<first turn>" },
  { "role": "assistant", "content": "<first answer>" },
  { "role": "user",      "content": "<current input>" }
]
```

No system message. `openagent-api` prepends the persona server-side. If the frontend accidentally sends one, `openagent-api` drops it with a warning log.
4. The list (plus optional `reasoning_effort`) is POSTed to `openagent-api`'s `/chat` endpoint with the `X-API-Key: OPENAGENT_API_KEY` header.
5. `openagent-api` prepends the persona, validates auth, and forwards to `openagent-infra`. `openagent-infra` injects the reasoning level and forwards to the BYOC provider.
6. The chunks flow back through `openagent-infra` and `openagent-api` byte-for-byte (SSE relay) and arrive at the frontend.
7. `sse_decoder.py` consumes the raw line iterator from `requests.iter_lines()` and yields typed `SSEEvent` objects:

- `kind="reasoning"` for chain-of-thought tokens (from `delta.reasoning`)
- `kind="content"` for visible answer tokens (from `delta.content`)
- `kind="finish"` for the `finish_reason` chunk
- `kind="error"` for in-band `[ERROR ...]` sentinels
- `kind="done"` for the `[DONE]` sentinel

8. `app.py`'s render loop routes each event to the right UI surface:
    - reasoning → live-streamed into a collapsible "Show thinking" expander
    - content → live-streamed into the main chat bubble
    - error → red banner via `st.error()`, loop breaks
    - done → loop breaks cleanly
9. When the stream completes, the assistant turn is appended to session state (without the reasoning — only `{role, content}` is persisted).

Separation of concerns

Concern	Lives in	Why
Chat UI rendering	`openagent-frontend`	Streamlit primitives, presentation only
In-session conversation state	`openagent-frontend`	UI cache
Reasoning-format display	`openagent-frontend`	UX decision (collapsible expander vs hidden vs inline)
Health gate (UI lock)	`openagent-frontend`	Lock pattern lives where the UI is rendered

Concern	Lives in	Why
Error display	`openagent-frontend`	Presentation; classification is upstream
Byte-level SSE / JSON parsing	`sse_decoder.py`	Isolated module so app.py doesn't import json
Persona / system prompt	`openagent-api`	Backend identity concern, not a UI concern
Messages list construction	`openagent-api`	Persona prepended server-side, single source of truth
Auth boundary to model layer	`openagent-api`	Frontend holds one key; gateway holds the chain
Upstream error normalisation	`openagent-api`	One classifier, not two; frontend trusts the codes
Model serving / inference	BYOC Provider	Heavy, GPU-dependent, handled by external compute
`Reasoning: <level>` injection	`openagent-infra`	Single source of truth for the prompt format

Tech Stack

Layer	Technology
Base image	`python:3.11-slim`
UI framework	Streamlit
HTTP client	`requests` (with `stream=True` for SSE)
Env loading	`python-dotenv`
Containerization	Docker + Docker Compose
Port (internal)	8501 (Streamlit default)
Port (host)	8000
Auth	API key via `X-API-Key` header
Communication	HTTP/1.1 + Server-Sent Events
Backend dependency	`openagent-api` (separate repo, port 8001)

Intentionally absent: `torch`, `transformers`, `accelerate`, `fastapi`, `httpx`, `sqlalchemy`, etc. None of them belong in a UI layer. See `requirements.txt` for the rationale.

Prerequisites

- **Docker Desktop** (macOS / Windows) or **Docker Engine + Compose v2** (Linux)
- `openagent-api` **running and reachable** — either on the host, in another Docker container, or deployed elsewhere
- **Valid API key** — the same value set as `OPENAGENT_API_KEY` in `openagent-api`'s `.env`

You do **not** need:

- A GPU (no inference happens here, or anywhere in the local stack)
- Python installed on the host (Docker handles it) — unless you want to run locally for development
- Compute provider API keys (those belong to `openagent-infra`)
- An `INFRA_API_KEY` value (that belongs to `openagent-api`)

The frontend only needs `OPENAGENT_API_KEY` — the secret for the frontend ↔ openagent-api boundary. The other two boundary keys (`INFRA_API_KEY` for openagent-api↔openagent-infra, `PROVIDER_API_KEY` for openagent-infra↔Provider) live in their respective services and never touch this repo.

Project Structure

```
openagent-frontend/
├── docker/
│   └── frontend/
│       └── Dockerfile              # Python 3.11 slim + Streamlit
├── src/
│   └── frontend/
│       ├── app.py                  # The Streamlit UI
│       └── sse_decoder.py          # SSE / ChatCompletion chunk decoder
├── docker-compose.yml              # Single-service compose
├── requirements.txt                # streamlit, requests, python-dotenv
├── .env                            # secrets — never commit this
├── .env.example                    # template for .env
├── .dockerignore                   # keeps .env and caches out of image
├── .gitignore                      # keeps .env and caches out of git
└── README.md                       # this file
```

Setup

1. Clone the repo

```
git clone https://github.com/william-mckeon/openagent-frontend.git
cd openagent-frontend
```

2. Create your `.env` file

```
cp .env.example .env
```

Open `.env` and set the two required values:

```
OPENAGENT_API_URL=http://host.docker.internal:8001
OPENAGENT_API_KEY=your_openagent_api_key_here
```

`OPENAGENT_API_KEY` MUST match the `OPENAGENT_API_KEY` value in `openagent-api`'s `.env` exactly. A mismatch produces `HTTP 401` on every `/chat` and `/health` call and surfaces in the UI as a 🔐 banner.

See Configuration for the full list of supported variables and when to use which `OPENAGENT_API_URL` value.

3. Make sure `openagent-api` is running

`openagent-frontend` is a thin client. Without `openagent-api` reachable at `OPENAGENT_API_URL`, the health gate sits on "🔌 Cannot reach openagent-api" and the chat input never unlocks.

Start `openagent-api` per its own README, then verify:

```
curl -H "X-API-Key: your_openagent_api_key_here"
http://localhost:8001/health
# {"status":"ok",...}           ← upstream warm, ready
# {"status":"loading",...}      ← compute worker spinning up
# {"status":"unreachable",...}  ← openagent-api can't reach openagent-
infra
```

Note: `/health` is authenticated. Without the `X-API-Key` header you'll get a 401 even from a fully-running gateway.

4. Build and start

```
docker-compose up -d --build
```

The `--build` flag is **required** any time `app.py`, `sse_decoder.py`, or the Dockerfile changes. Without it Docker reuses the cached image.

First build takes 1–2 minutes (pip install layer). Subsequent builds are sub-second thanks to Docker layer caching.

5. Open in browser

```
http://localhost:8000
```

On first load you will see one of four states:

- 🟢 **"openagent-api ready — starting chat"** — upstream is warm, chat input is live
- ⏳ **"The upstream model is starting up"** — cold start; the page polls every 3 seconds and unlocks automatically when ready
- 🔌 **"openagent-api is up but cannot reach the upstream model"** — gateway is fine, openagent-infra or compute provider is down
- 🔌 **"Cannot reach openagent-api at …"** — gateway unreachable; fix `OPENAGENT_API_URL` in `.env` and retry

6. (Optional) Tail logs

```
docker-compose logs -f openagent-frontend
```

Logs use the same format as `openagent-api` and `openagent-infra` so lines align when tailing all three services simultaneously. The frontend's named logger is `openagent.frontend`, with a child `openagent.frontend.sse_decoder` for the decoder module.

How It Works

The frontend owns presentation, not identity

The persona is owned by `openagent-api` — the frontend has no copy, no path, and no system prompt file. It just sends user/assistant turns and trusts the gateway to do the right thing upstream.

This means:

- Any client — a mobile app, a CLI — gets the same agent identity by talking to `openagent-api`, with no need to re-implement persona ownership
- A tampered or out-of-date frontend cannot override the persona
- The frontend stays out of the business of holding configuration that has no place at the UI layer

Conversation history (client-side)

`openagent-api` is stateless across requests. Every `/chat` call must include the full history. `openagent-frontend` holds the history in `st.session_state.messages` and sends the complete list on every turn.

The reasoning chain streams live during generation and is rendered in the expander, but it isn't stored back into history. The schema is clean OpenAI shape: `{"role", "content"}`.

Health gate

During a serverless cold start `openagent-infra` reports `{"status": "degraded"}`, which `openagent-api` translates to `{"status": "loading"}` for the frontend.

To prevent users from firing messages that would just 503, `openagent-frontend` implements a **blocking health gate**:

1. On every Streamlit rerun, if `session_state.model_ready` is `False`, a `while` loop polls `/health` every 3 seconds with the `X-API-Key` header.
2. A live status banner updates based on the response's top-level `status` field:
   - `ok` → flip `model_ready`, show 🟢 briefly, `st.rerun()` to load the chat UI
   - `loading` → show ⏳ with cold-start narrative and attempt counter
   - `unreachable` → show 🔌 with "openagent-api is up but cannot reach the upstream model"
   - Connection error → show 🔌 with the URL and retry info
   - Anything else → show ⚠ with the raw status
3. The chat UI is literally not rendered until the gate passes.

The frontend doesn't need to know about the compute provider or openagent-infra; `openagent-api` translates upstream vocabulary into a tidy three-value response.

SSE streaming with JSON ChatCompletion chunks

The upstream emits OpenAI ChatCompletion chunks. Each event is JSON-encoded, with chain-of-thought tokens in `choices[0].delta.reasoning` and visible answer tokens in `choices[0].delta.content`. The two streams interleave only at chunk boundaries.

The decoding lives in `src/frontend/sse_decoder.py`:

1. Consumes raw lines from `response.iter_lines(decode_unicode=True)`
2. Skips blank lines and SSE comments silently
3. Strips the `data:` prefix
4. Detects the two non-JSON sentinels (`[DONE]` and `[ERROR upstream_status=...]`) before attempting JSON parse
5. JSON-decodes everything else as a ChatCompletion chunk
6. Routes by `delta` key: `reasoning` → `SSEEvent("reasoning")`, `content` → `SSEEvent("content")`, `finish_reason` set → `SSEEvent("finish")`
7. Skips malformed chunks with a `WARNING` log
8. Yields typed `SSEEvent` dataclasses to `app.py`

`app.py`'s render loop then routes each event by `event.kind` — reasoning into the expander, content into the chat bubble, error into a red banner, done into a clean break.

Error handling

The frontend exposes a consistent error taxonomy with emoji prefixes so I can scan logs and banners at a glance.

Prefix	Class	Trigger

Prefix	Class	Trigger
🔌	Connection / network	TCP connect failed, `openagent-api` unreachable, HTTP 502
⏳	Timeout / model loading	Connect timeout, HTTP 503 (model loading), HTTP 504 (upstream timeout)
🔐	Auth	HTTP 401 — `OPENAGENT_API_KEY` mismatch
⚠	Request validation	HTTP 400 (empty messages) or HTTP 422 (invalid `reasoning_effort`)
❌	Unexpected	Anything else (parse errors, unexpected exceptions)

Mid-stream errors arrive as in-band SSE events (`data: [ERROR upstream_status=503]`) which `sse_decoder.py` recognises and yields as `SSEEvent(kind="error", error=...)`. The render loop displays the banner and stops consuming.

On any error, the partial response is **not** appended to history.

Configuration

All configuration is loaded from `.env` at the repository root via `python-dotenv` and `docker-compose`'s `env_file:` directive. See `.env.example` for the template.

Variable	Type	Default	Description
`OPENAGENT_API_URL`	string	`http://localhost:8001`	Base URL of `openagent-api`. No trailing slash.
`OPENAGENT_API_KEY`	string	—	Shared secret for `X-API-Key` header on `/chat` and `/health`. Required.

Choosing the right `OPENAGENT_API_URL`

Scenario	Value
Everything on host, no Docker	`http://localhost:8001`
Frontend in Docker, `openagent-api` on host	`http://host.docker.internal:8001`
Both in Docker on a shared external network	`http://openagent-api:8001`
External deployment	`https://api.your-domain.com`

The compose file declares `extra_hosts: host.docker.internal:host-gateway` so `host.docker.internal` resolves correctly on Linux as well as Docker Desktop.

The three-key compartmentalization model

`OPENAGENT_API_KEY` in this file is **only** the frontend ↔ openagent-api boundary key. It is not shared with the model layer. The full picture:

Boundary	Key	Lives in
frontend ↔ openagent-api	`OPENAGENT_API_KEY`	`openagent-frontend/.env` + `openagent-api/.env`
openagent-api ↔ infra	`INFRA_API_KEY`	`openagent-api/.env` + `openagent-infra/.env`
infra ↔ Provider	`PROVIDER_API_KEY`	`openagent-infra/.env`

Each pair of services has its own shared secret. No key is forwarded unchanged through the chain.

Generating a new API key

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Paste the output into **both** `openagent-api/.env` (as `OPENAGENT_API_KEY=…`) **and** `openagent-frontend/.env` (as `OPENAGENT_API_KEY=…`). The two must match byte-for-byte.

Local Development (without Docker)

Sometimes you want faster iteration than a Docker rebuild. Run Streamlit directly on the host:

```
# 1. Create a virtualenv
python3.11 -m venv .venv
source .venv/bin/activate   # Linux/macOS
# .venv\Scripts\activate    # Windows

# 2. Install dependencies
pip install -r requirements.txt

# 3. Export env or rely on .env (python-dotenv picks it up)
export OPENAGENT_API_URL=http://localhost:8001
export OPENAGENT_API_KEY=your_openagent_api_key_here

# 4. Run Streamlit on port 8000
streamlit run src/frontend/app.py \
  --server.port 8000 \
  --server.address 0.0.0.0 \
  --server.headless true \
  --server.fileWatcherType none
```

Streamlit supports hot-reload — edit `app.py` or `sse_decoder.py` and the browser refreshes automatically.

Design Decisions

Why Streamlit?

Streamlit collapses "build a chat UI, style it, add streaming, manage session state, serve it over HTTP" into a single Python file with no JavaScript. The HTTP/SSE contract with `openagent-api` means the frontend can be swapped wholesale (mobile app, CLI, alternate web framework) without touching the backend later.

Why is the SSE decoder a separate module?

1. **Single responsibility.** It isolates JSON-decoding-and-event-routing from Streamlit-and-rendering.
2. **Testability.** `parse_chunk()` takes a string and returns an event, unit-testable in isolation.
3. **Containment of upstream changes.** When the chunk format evolves, the change is contained to one file.

Why a blocking health-polling loop?

Streamlit is single-threaded and lacks native auto-refresh. A blocking `while` loop with `st.empty()` status updates is the simplest correct pattern that keeps the page responsive (live status updates) and prevents users from firing requests that would 503.

Why does the frontend trust openagent-api's error normalisation?

Because `openagent-api` owns the upstream relationship. Re-classifying errors at the frontend would mean duplicating logic that already exists upstream. `openagent-api` maps upstream conditions onto a small set of HTTP status codes; the frontend just maps each code to an emoji prefix and a message.

Why pure pass-through on `reasoning_effort`?

One source of truth. `openagent-infra` holds the default; `openagent-api` passes through; the frontend either sets a value or omits the field. Adding a frontend-side default would create two places to check when debugging.

Why the emoji error-prefix scheme (🔌 ⏳ 🔐 ⚠❌)?

Consistency. Once you've learned the error semantics in one part of the stack, you can read any log without a legend, and the prefixes are greppable.

Why port 8501 internal and 8000 external?

8501 is Streamlit's default — keeping it as the container's internal port means zero Streamlit config overrides. 8000 is the user-facing port. The mapping happens in `docker-compose.yml` where it belongs.

Why Python 3.11 slim?

Matches `openagent-api`'s base image for consistency across the stack. Slim variant keeps the image around 450 MB total. No CUDA, no BLAS, no compilers needed — this is a pure-Python HTTP client.

Known Limitations

These are present-tense limitations I'm aware of and have chosen to live with for now.

No per-user identity

`OPENAGENT_API_KEY` is a single shared secret, not a per-user credential. There is no concept of "user A vs user B" at this layer — anyone holding the key gets full access to whatever the frontend can do. Fine for trusted, single-operator use; not appropriate for public exposure without an auth layer in front.

No rate limiting or abuse protection

The frontend trusts its users. If it were exposed to a wider audience, rate limiting would belong at `openagent-api` or at a reverse proxy in front of it — not at the UI layer.

Single-user, single-browser session

Conversation state lives in `st.session_state`, which is per browser tab. There is no cross-session persistence; closing the tab loses the history.

No context-window truncation

The frontend forwards whatever message list it has accumulated. A long enough conversation will eventually exceed the upstream model's context window and the request will fail upstream. There is no client-side summarisation or trimming.

Cold-start UX is a blocking wait

During the initial serverless worker spin-up, users see a live status banner but cannot do anything else. Acceptable for a personal tool; a "come back later" UX would be more appropriate for a public-facing deployment. Nothing the frontend does can speed up the worker spin-up — the wait is at the inference layer.

Troubleshooting

🔌 "Cannot reach openagent-api at …"

The frontend cannot establish a TCP connection to `OPENAGENT_API_URL`. Check:

1. Is `openagent-api` running? `curl -H "X-API-Key: <your key>" http://localhost:8001/health` from the host.
2. Is `OPENAGENT_API_URL` in `.env` correct for your topology?
3. On Linux, is `extra_hosts: host.docker.internal:host-gateway` doing its job? Try `docker exec openagent-frontend getent hosts host.docker.internal`.

🔌 "openagent-api is up but cannot reach the upstream model"

`openagent-api` is reachable but its `/health` returned `{"status": "unreachable"}`. Debug at `openagent-api` and `openagent-infra`. Check:

1. Is `openagent-infra` running? Is its URL in `openagent-api/.env` correct?
2. Are provider credentials valid in `openagent-infra/.env`?
3. Tail `openagent-api` logs: `docker-compose -p openagent-api logs -f`

🔐 "API key missing or invalid"

`OPENAGENT_API_KEY` in `openagent-frontend/.env` does not match `OPENAGENT_API_KEY` in `openagent-api/.env`. Copy the exact value across and restart both containers.

⏳ "The upstream model is starting up"

Normal on serverless cold starts. The serverless worker scales to zero when idle. The gate clears automatically once `openagent-api` reports `status: ok`.

"I see reasoning tokens in the main bubble"

If chain-of-thought text appears in the answer area instead of the expander, `sse_decoder.py` is misclassifying chunks. The upstream model may be putting CoT in `delta.content` directly.

Changes to `app.py` or `sse_decoder.py` not taking effect

The Docker image bakes in source code at build time. Use `docker-compose up -d --build` to rebuild with the changes, OR run locally without Docker for hot-reload during iteration.

License

Maintainer

William McKeon (github.com/william-mckeon)